

JC Iturria Lazarini

Ali Khanchi

Tom Kuntkes

Ruben Sterzing



university of
 groningen



Agenda



Agenda

Phase 1



Agenda

Phase 2

5

Continuous
integration

4

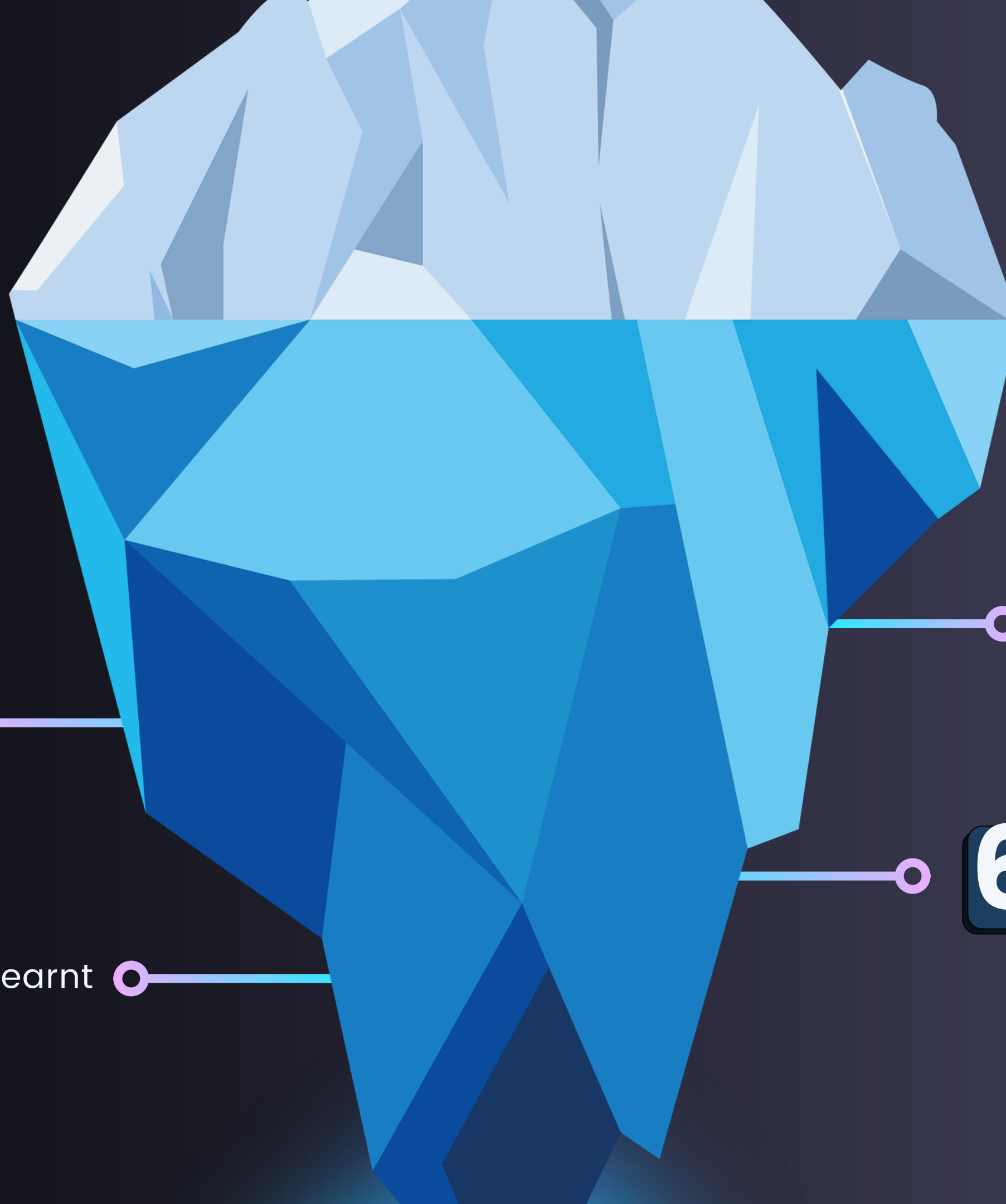
Technology
Stack

6

Architectural
structure

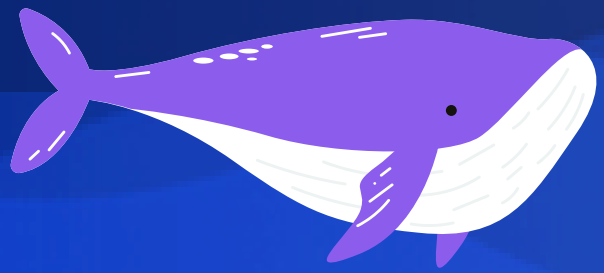
7

Lessons learnt



PHase

ONE



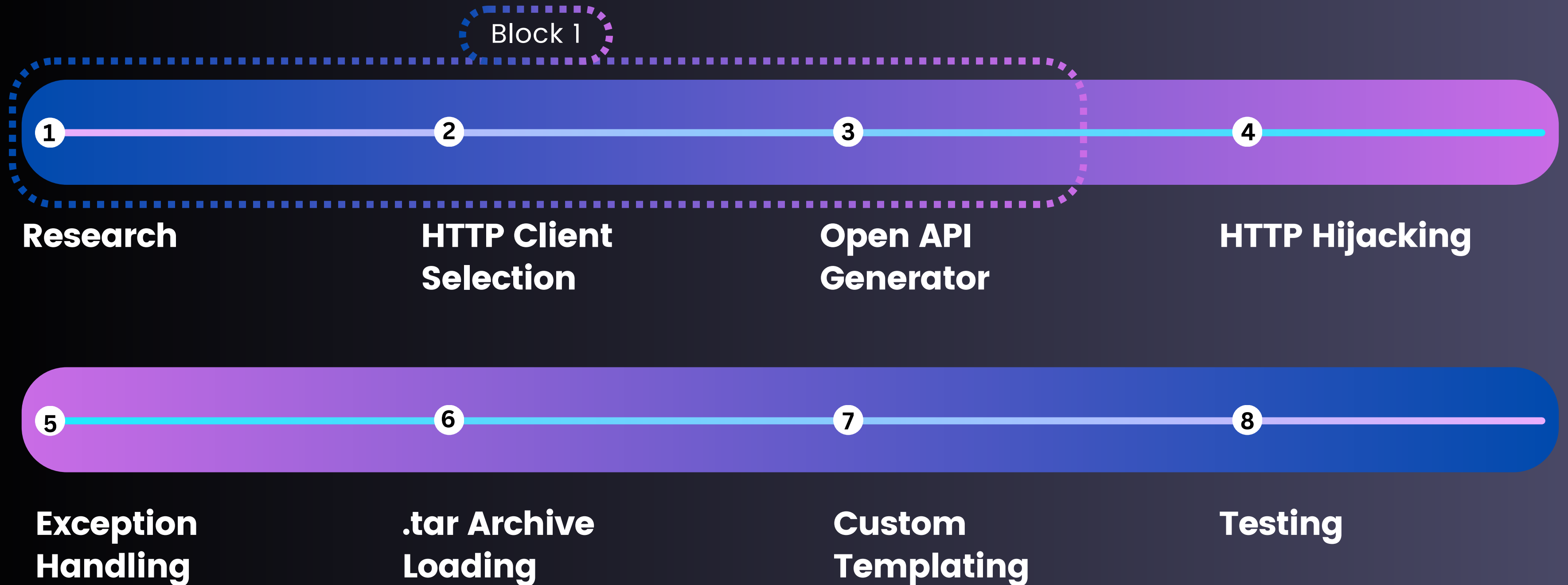
Recap

What is Docker Kotlin



A modern Docker Library in Kotlin

Plan & Structure



Client Communication

1 Slack

2 Weekly meetings

3 Gitlab

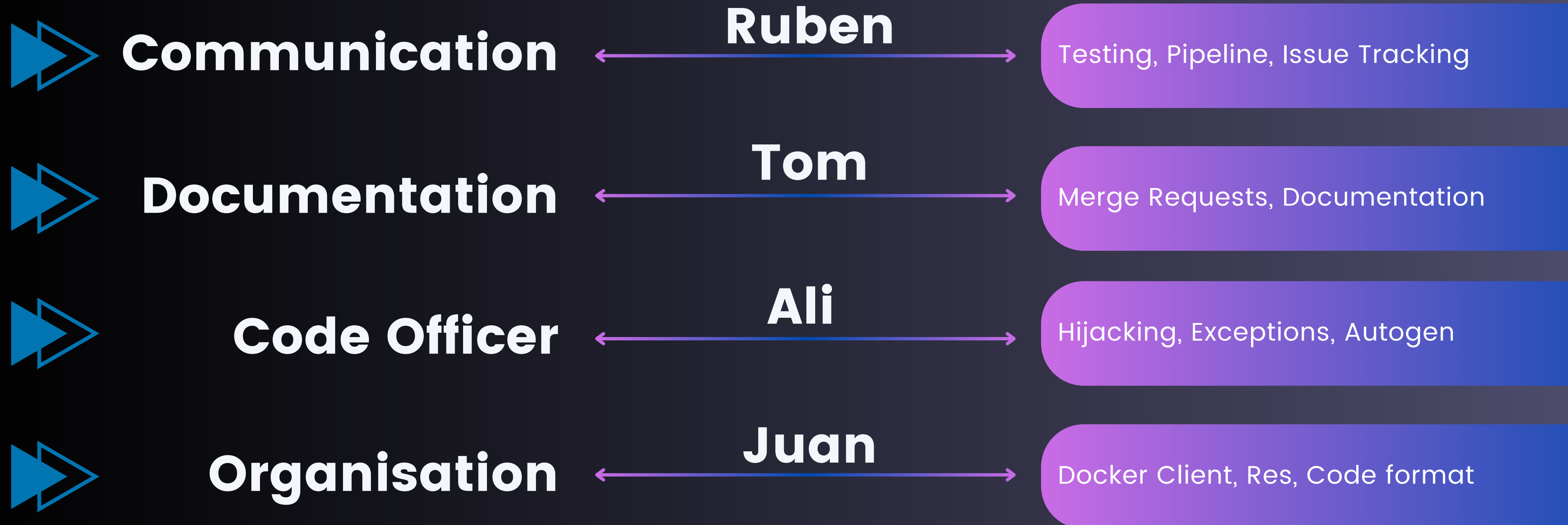


Team Communication

- 1 **Campus**
- 2 **Discord**
- 3 **Agile method**
- 4 **Taiga/Kanban**



Responsibilities



Best Practices

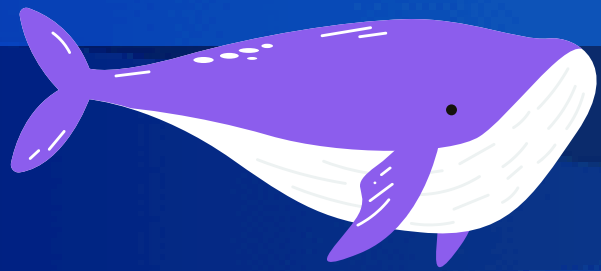
**Code Reviews on
Merge Request**

Issue Tracking

**Client Feedback
and Discussion**

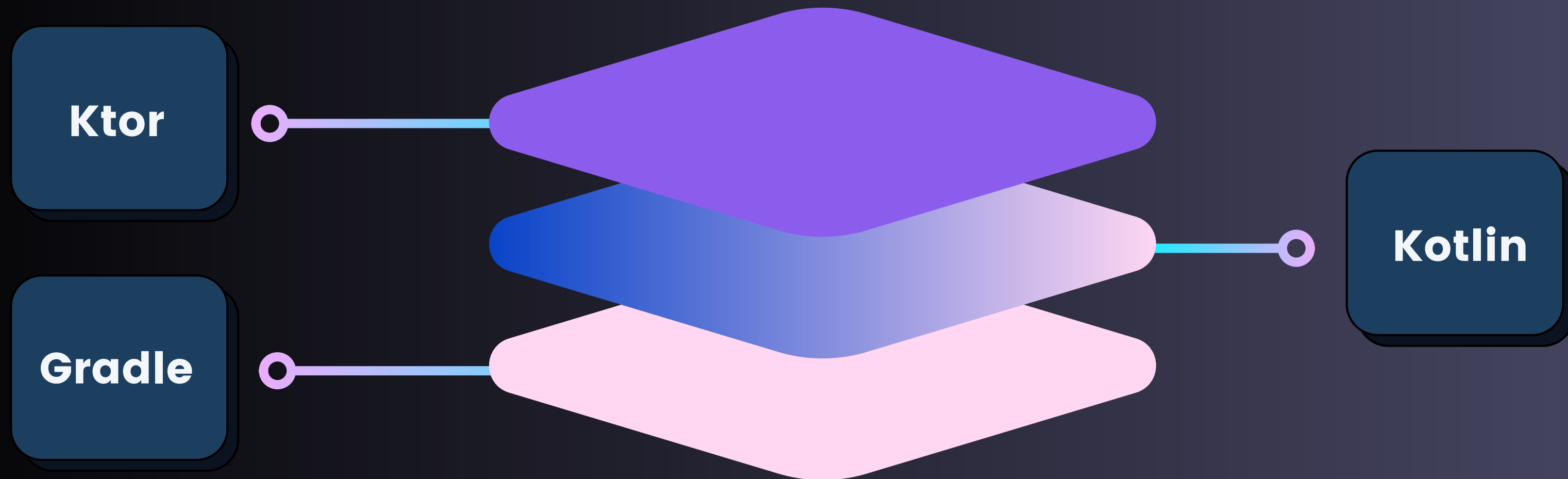
PHase

TWO



Technology Stack

Docker Kotlin



Continuous Integration



Statistics

☆ test PUBLIC

✓ Passed

Last analysis: 14 minutes ago · 1.9k Lines of Code · Kotlin

 0	 1	 18	 —	 26.8%	 0.0%
Security	Reliability	Maintainability	Hotspots Reviewed	Coverage	Duplications

Lines Committed

2.3K

Merge Requests

4

- Completed

1

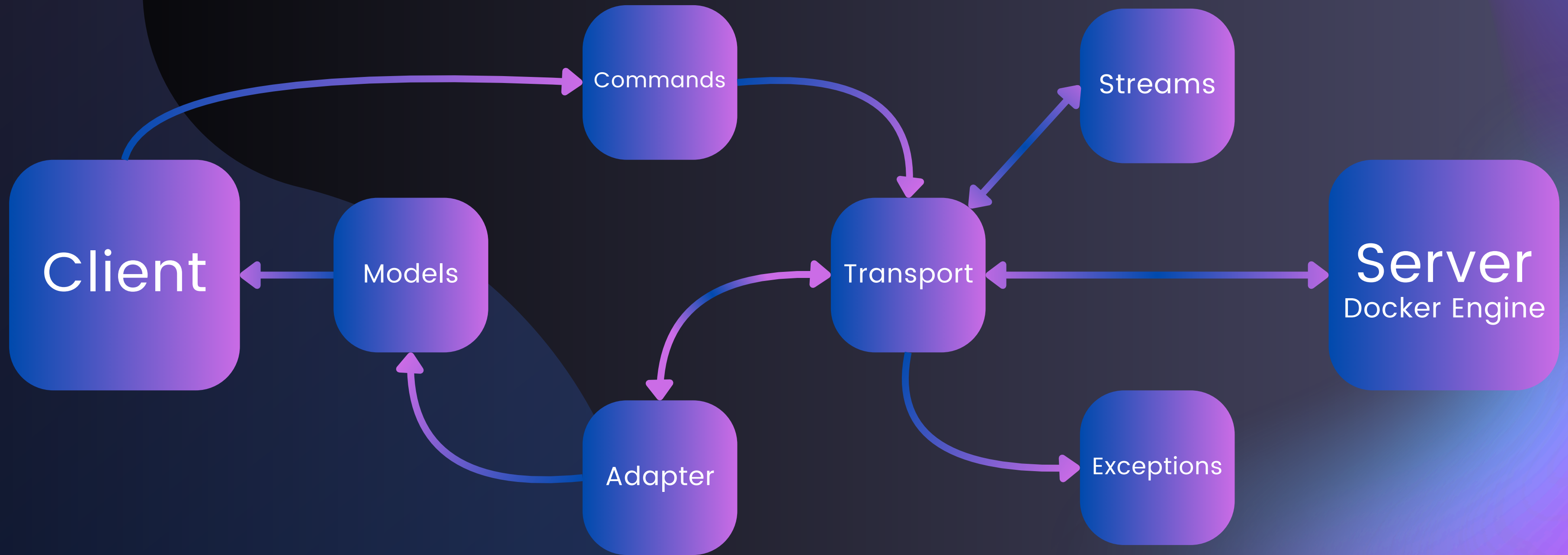
- Merged to Develop

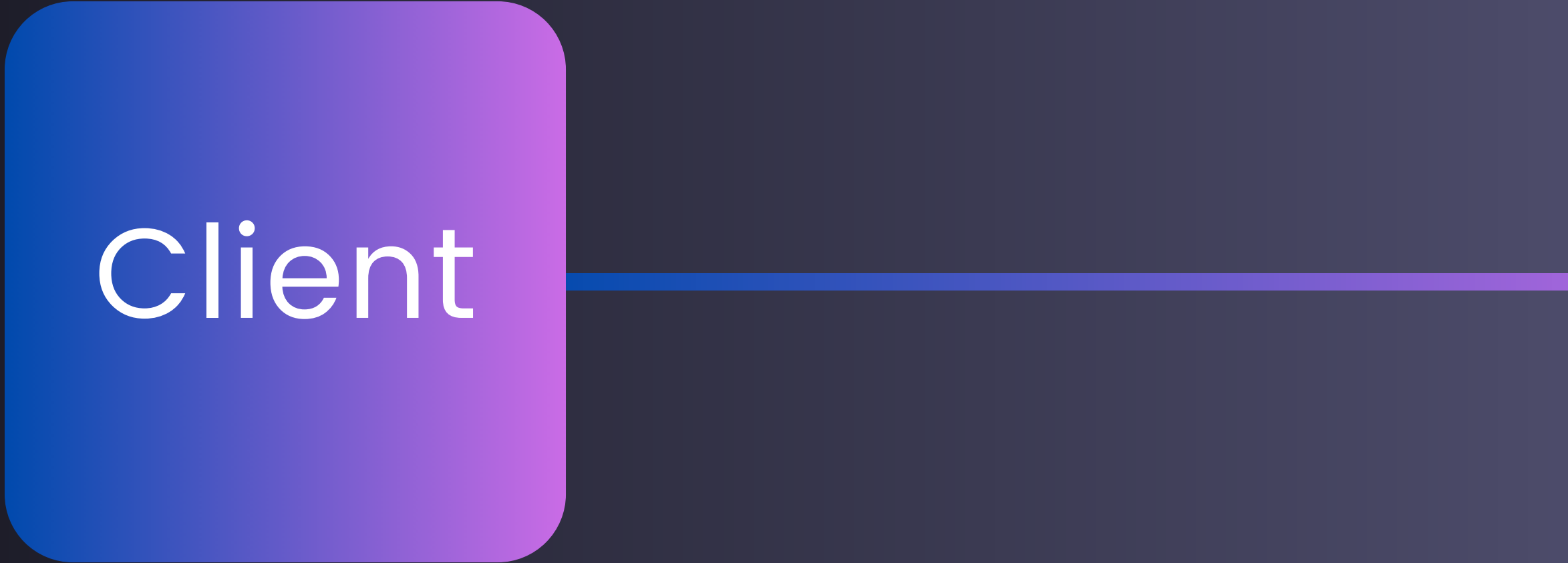
Code

9/10 Main Endpoints

26.8% Coverage

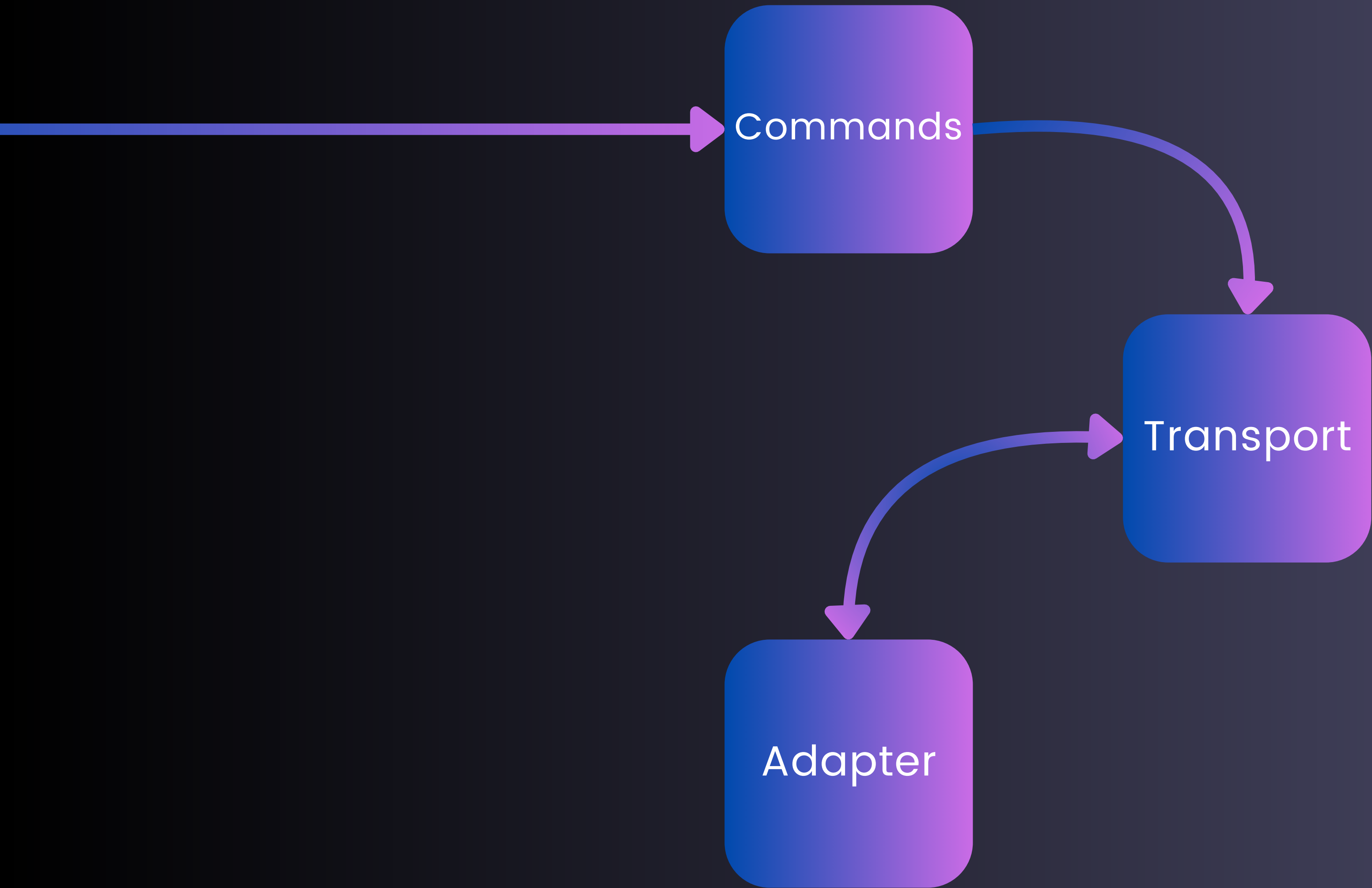
Architectural Structure

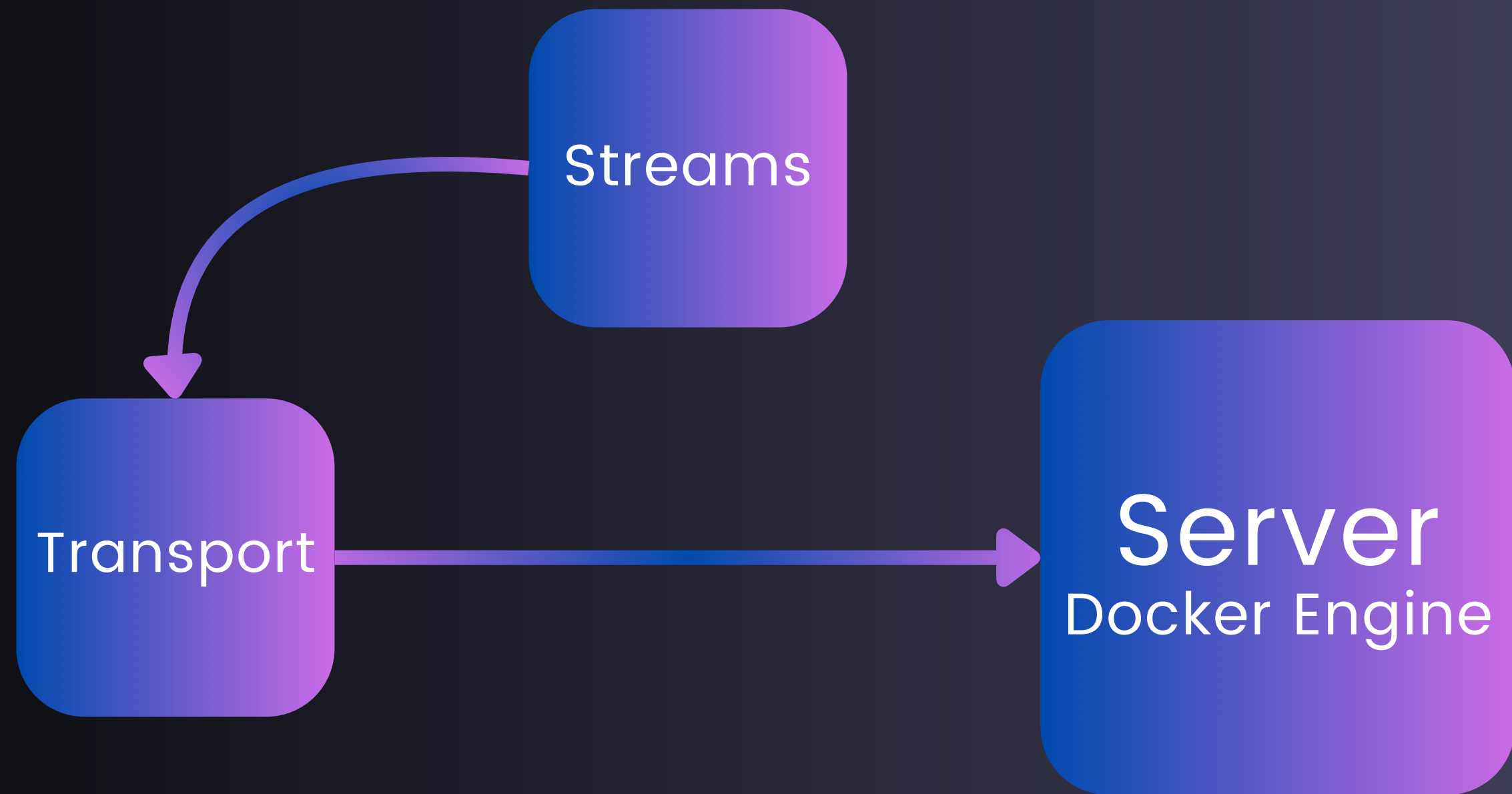


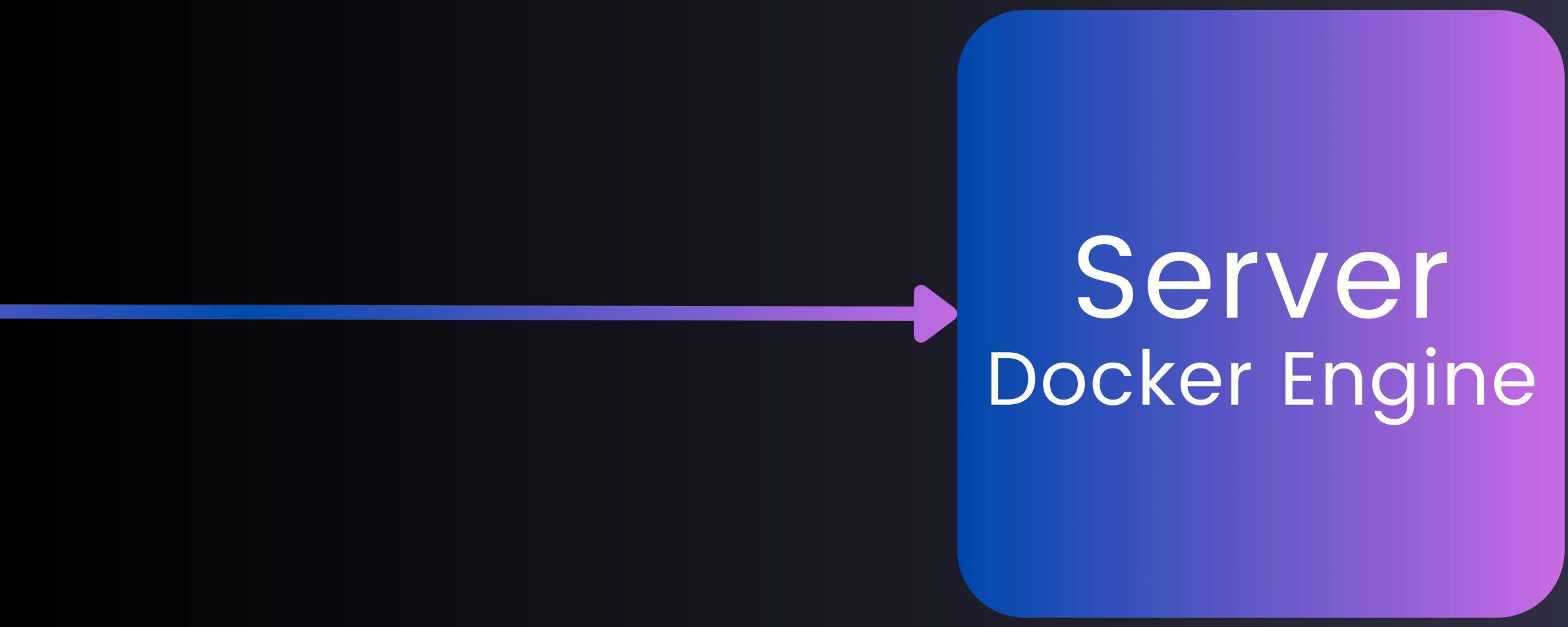


Client

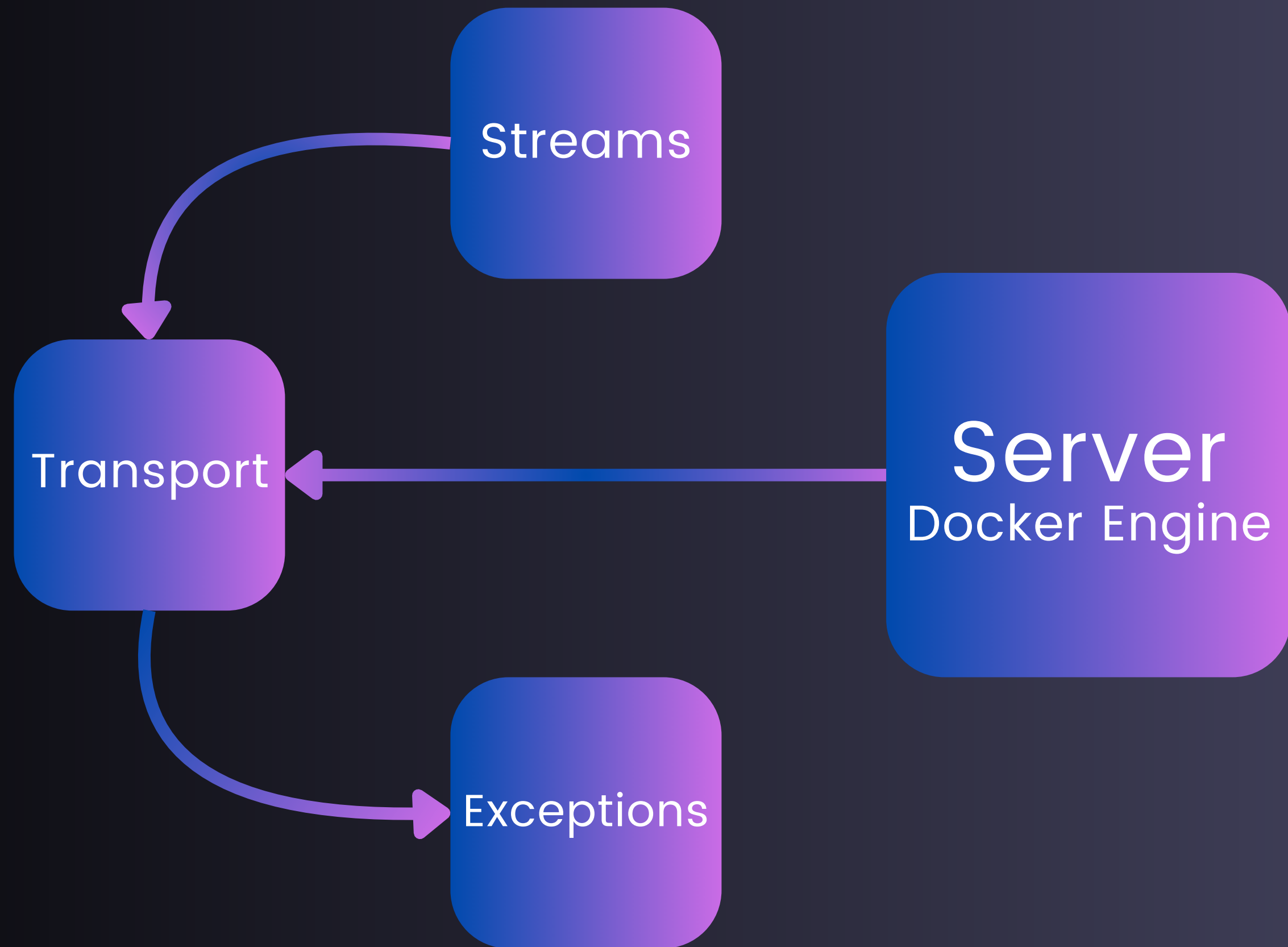
The image features a dark blue background. In the center-left, there is a rounded square box with a horizontal gradient from blue on the left to purple on the right. The word 'Client' is written in white, sans-serif font inside this box. A thin horizontal line, also with a blue-to-purple gradient, extends from the right side of the box across the entire width of the image.

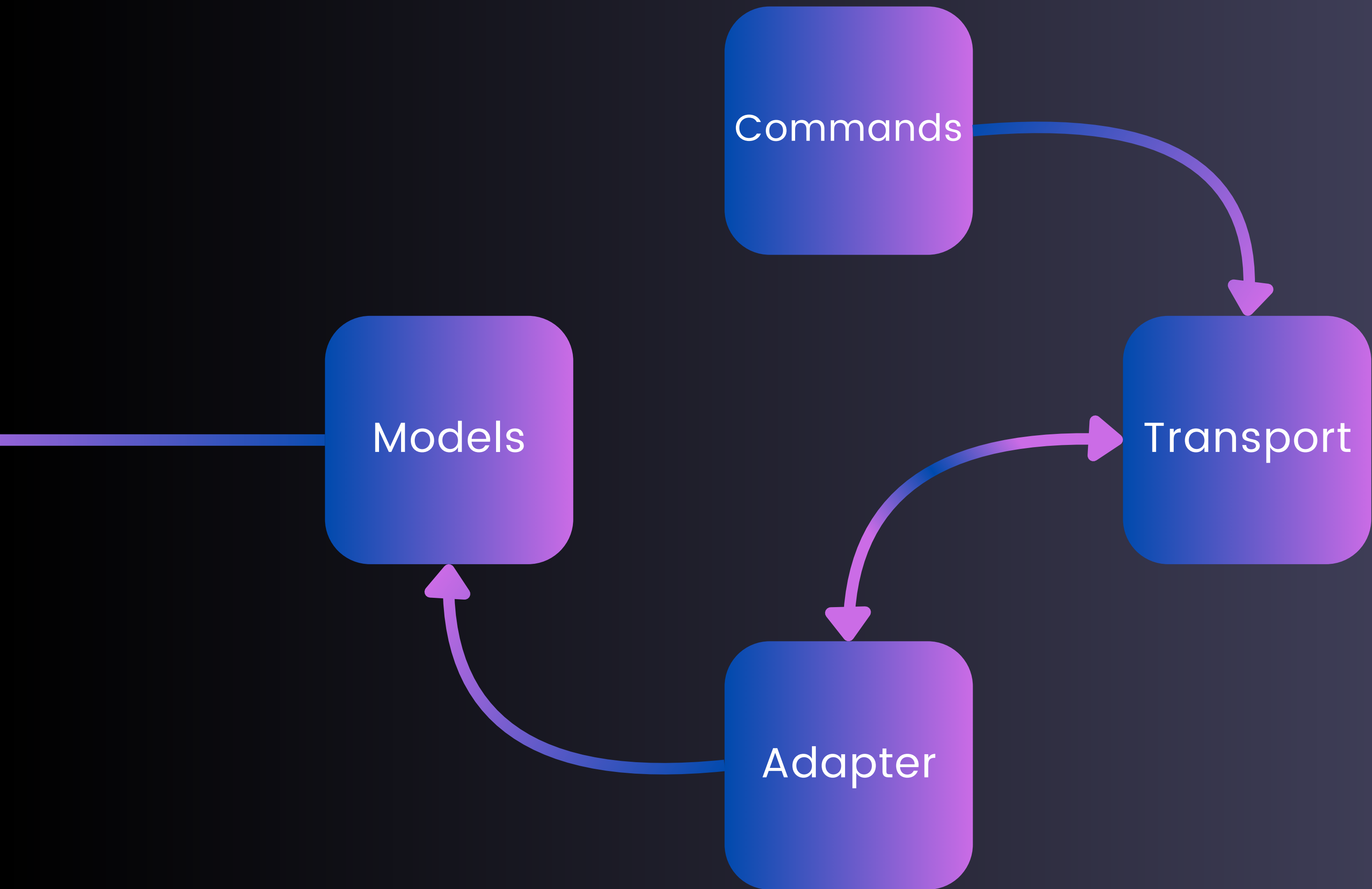




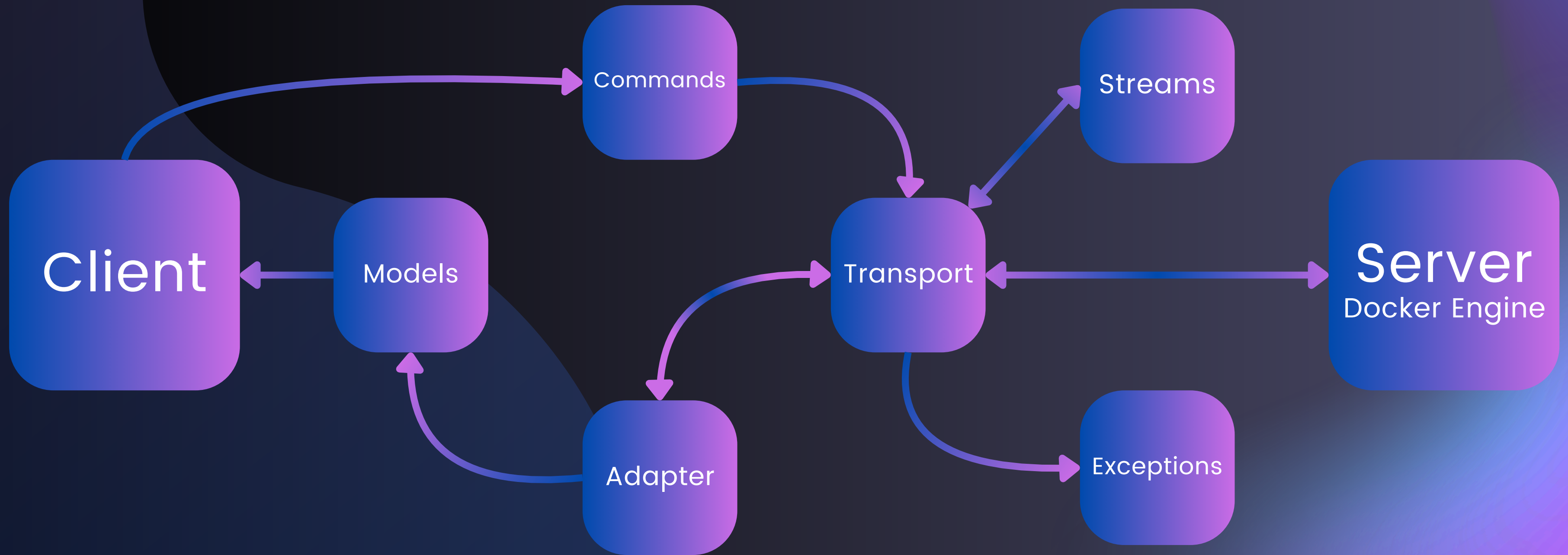


Server
Docker Engine





Architectural Structure



Aspects we're most proud of



AUTOGEN

“The process is automated and part of the build process of the project itself, which is what we like a lot.”



EXCEPTIONS

“The exception system within the docker library is quite extensive. We are very satisfied with this.”

BEFORE

```
fun main() {  
    val config = DefaultDockerClientConfig.createDefaultConfigBuilder()  
        .withDockerHost(dockerHost: "unix:///var/run/docker.sock")  
        .build()  
    val httpClient: DockerHttpClient = ApacheDockerHttpClient.Builder()  
        .dockerHost(config.dockerHost)  
        .connectionTimeout(Duration.ofSeconds(seconds: 20))  
        .build()  
    val dj = DockerClientImpl.getInstance(config, httpClient)  
    val containerName = "grindTest"  
    dj.copyArchiveToContainerCmd(containerName).withContainerId(containerName)  
        .withRemotePath("/")  
        .withTarInputStream(File(pathname: "valDemo.tar").inputStream())  
        .withNoOverwriteDirNonDir(true).exec()  
}
```

DOCKER-JAVA

AFTER

```
suspend fun main() {  
    val baseUrl = "unix:///var/run/docker.sock"  
    val dk = DockerClient(baseUrl, {  
        developmentMode = true  
    }, {  
        requestTimeout = 20e3.toLong()  
    })  
    val containerName = "grindTest"  
    dk.container.putArchive(containerName, noOverwriteDirNonDir = "1",  
        inputStream = File(pathname: "valDemo.tar"), path = "/")  
}
```

DOCKER-KOTLIN

KTOR

```
val localVariableAuthNames = listOf<String>()
val localVariableBody = body
val localVariableQuery = mutableMapOf<String, List<String>>()
name?.apply { localVariableQuery["name"] = listOf("$name") }
platform?.apply { localVariableQuery["platform"] = listOf("$platform") }
val localVariableHeaders = mutableMapOf<String, String>()
val localVariableConfig = RequestConfig<kotlin.Any?>(
    RequestMethod.POST, path: "/containers/create", query = localVariableQuery,
    headers = localVariableHeaders, requiresAuthentication = false,)
return client.request(localVariableConfig, localVariableBody, localVariableAuthNames).wrap()
```

```
client.request {
    url {
        takeFrom(URLBuilder(protocol = URLProtocol(name: "unix", port)))
        appendPath(requestConfig.path.trimStart(...chars: '/').split(...delimiters: '/'))
        requestConfig.query.forEach { query ->
            query.value.forEach { value ->
                parameter(query.key, value)
            }
        }
    }
    method = requestConfig.method.httpMethod
    headers.filter { header -> !UNSAFE_HEADERS.contains(header.key) }
        .forEach { header -> this.header(header.key, header.value) }
    if (requestConfig.method in listOf(RequestMethod.PUT, RequestMethod.POST, RequestMethod.PATCH)) {
        val contentType = (requestConfig.headers[HttpHeaders.ContentType]?.let { ContentType.parse(it) }
            ?: ContentType.Application.Json)
        this.contentType(contentType)
        setBody(body)
    }
    attributes.put(AttributeKey(name: "address"), baseUrl.substringAfter(delimiter: "://"))
    attributeMap?.apply { attributes.putAll(other: this) }
}
```

KTOR

```
val localVariableAuthNames = listOf<String>()
val localVariableBody = body
val localVariableQuery = mutableMapOf<String, List<String>>()
name?.apply { localVariableQuery["name"] = listOf("$name") }
platform?.apply { localVariableQuery["platform"] = listOf("$platform") }
val localVariableHeaders = mutableMapOf<String, String>()
val localVariableConfig = RequestConfig<kotlin.Any?>(
    RequestMethod.POST, path: "/containers/create", query = localVariableQuery,
    headers = localVariableHeaders, requiresAuthentication = false,)
return client.request(localVariableConfig, localVariableBody, localVariableAuthNames).wrap()
```

```
client.request {
    url {
        takeFrom(UrlBuilder(protocol = URLProtocol( name: "unix", port)))
        appendPath(requestConfig.path.trimStart(...chars: '/').split(...delimiters: '/'))
        requestConfig.query.forEach { query ->
            query.value.forEach { value ->
                parameter(query.key, value)
            }
        }
    }
    method = requestConfig.method.httpMethod
    headers.filter { header -> !UNSAFE_HEADERS.contains(header.key) }
        .forEach { header -> this.header(header.key, header.value) }
    if (requestConfig.method in listOf(RequestMethod.PUT, RequestMethod.POST, RequestMethod.PATCH)) {
        val contentType = (requestConfig.headers[HttpHeaders.ContentType]?.let { ContentType.parse(it) }
            ?: ContentType.Application.Json)
        this.contentType(contentType)
        setBody(body)
    }
    attributes.put(AttributeKey( name: "address"), baseUrl.substringAfter( delimiter: "://"))
    attributeMap?.apply { attributes.putAll( other: this) }
}
```

```
val baseUrl = "unix:///var/run/docker.sock"
val dk = DockerClient(baseUrl, {
    developmentMode = true
}, {
    requestTimeout = 20e3.toLong()
})
dk.container.create(ContainerCreateRequest(image = "alpine", tty = true,
    openStdin = true), name: "testContainer")
```

DOCKER-KOTLIN

Lessons Learnt



Accountability



Dividing Tasks



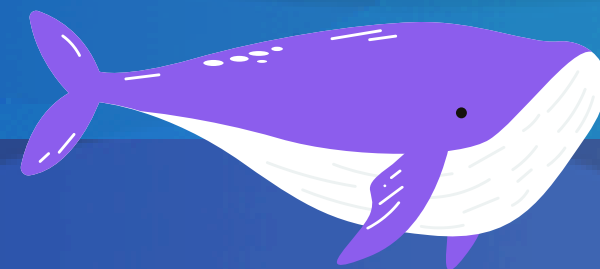
Consult Client

Features

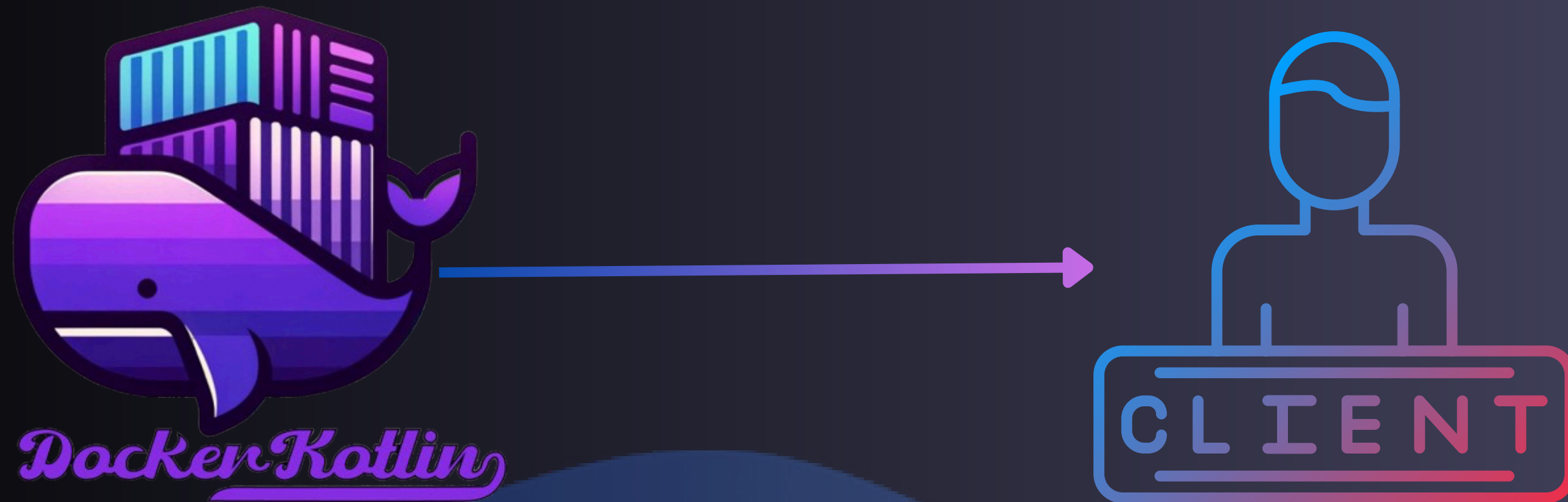
CLIENT

- ✓ Basic Endpoints Covered
- ✓ Most Used Endpoints
- ✓ Asynchronous calls
- ✓ Auto Generation in Build
- ✓ Applied Patterns
- ✓ Extensive Exceptions
- 1/2 HTTP Hijacking

LIVE DEMO



Project Handover



Thank you!

Questions?